

Практика 5. Типы данных

Андрей Стоян, Илья Колегов, Дмитрий Халанский

MSE ITMO

осень 2025

В предыдущих сериях

- ▶ Структуры данных в чистом лямбда-исчислении
- ▶ Базовый синтаксис Haskell
- ▶ Базовые типы Haskell
- + Списки работа с ними
- + Алгебраические типы данных и сопоставление с образцом

Содержание

1. data
2. Полиморфные data
3. Списки
4. Всякое разное (⊕)
5. Алгебраические типы данных в других языках (☀)
6. Материалы (☀)

Содержание

1. data
2. Полиморфные data
3. Списки
4. Всякое разное (⊕)
5. Алгебраические типы данных в других языках (☀)
6. Материалы (☀)

Зачем встроенные структуры данных в Haskell

- ▶ В λ -исчислении мы уже пользовались всеми необходимыми структурами данных
 - ▶ Пары: **pair**, **fst**, **snd**
 - ▶ Варианты: **inl**, **inr**, **either**
 - ▶ Числа: **zero**, **suc**, **natElim**
 - ▶ Списки: **nil**, **cons**, **fold**
- ▶ Однако эти реализации не всегда годятся на практике:
 - ▶ Элиминация часто требует обхода всей структуры
 - ▶ Слишком сложно типизируются
- ▶ Haskell имеет встроенную поддержку структур данных
 - ▶ Эффективно представляются на уровне значений
 - ▶ Просто и номинативно типизируются

Конструирование производений

Декларация типа-произведения

```
data CatT = CatD String Int
```

Конструирование производений

Декларация типа-произведения

```
data CatT = CatD String Int
```

- ▶ CatT — имя типа (типовой константы)
- ▶ CatD — конструктор данных: `CatD :: String -> Int -> CatT`
- ▶ CatD — специальная функция, которая реализуется в самом языке
- ▶ Имена типов и конструкторов данных не пересекаются — их можно называть одинаково

Конструирование производений

Декларация типа-произведения

```
data CatT = CatD String Int
```

- ▶ CatT — имя типа (типовой константы)
- ▶ CatD — конструктор данных: `CatD :: String -> Int -> CatT`
- ▶ CatD — специальная функция, которая реализуется в самом языке
- ▶ Имена типов и конструкторов данных не пересекаются — их можно называть одинаково

Конструирование

```
barsik :: CatT  
barsik = CatD "barsik" 42
```


Конструирование произведений

Декларация типа-произведения

```
data CatT = CatD String Int
```

- ▶ CatT — имя типа (типовой константы)
- ▶ CatD — конструктор данных: `CatD :: String -> Int -> CatT`
- ▶ CatD — специальная функция, которая реализуется в самом языке
- ▶ Имена типов и конструкторов данных не пересекаются — их можно называть одинаково

Конструирование

```
barsik :: CatT  
barsik = CatD "barsik" 42
```

Количество значений

$$|CatT| = |String| * |Int|$$

Конструирование сумм

Декларация типа-суммы

```
data Pet = Cat CatT | Dog String Int | Parrot
```

Конструирование сумм

Декларация типа-суммы

```
data Pet = Cat CatT | Dog String Int | Parrot
```

- ▶ Значение типа `Pet` можно конструировать несколькими различными способами
- ▶ `Cat :: CatT -> Pet`
- ▶ `Dog :: String -> Int -> Pet`
- ▶ `Parrot :: Pet`

Конструирование сумм

Декларация типа-суммы

```
data Pet = Cat CatT | Dog String Int | Parrot
```

- ▶ Значение типа Pet можно конструировать несколькими различными способами
- ▶ `Cat :: CatT -> Pet`
- ▶ `Dog :: String -> Int -> Pet`
- ▶ `Parrot :: Pet`

Конструирование

```
petCat :: CatT -> Pet
```

```
petCat cat = Cat cat
```

Конструирование сумм

Декларация типа-суммы

```
data Pet = Cat CatT | Dog String Int | Parrot
```

- ▶ Значение типа Pet можно конструировать несколькими различными способами
- ▶ `Cat :: CatT -> Pet`
- ▶ `Dog :: String -> Int -> Pet`
- ▶ `Parrot :: Pet`

Конструирование

```
petCat :: CatT -> Pet
```

```
petCat cat = Cat cat
```

Количество значений

$$|\text{Pet}| = |\text{CatT}| + |\text{String}| * |\text{Int}| + 1$$

Элиминация алгебраических типов

Декларации

```
data CatT = CatD String Int
```

```
data Pet = Cat CatT | Dog String Int | Parrot
```

Элиминация алгебраических типов

Декларации

```
data CatT = CatD String Int
data Pet  = Cat CatT | Dog String Int | Parrot
```

Паттерн-матчинг (сопоставление с образцом)

Элиминация структуры данных путём сопоставления её со способом конструирования.

```
show :: Pet -> String
show pet = case pet of
```

Элиминация алгебраических типов

Декларации

```
data CatT = CatD String Int
data Pet = Cat CatT | Dog String Int | Parrot
```

Паттерн-матчинг (сопоставление с образцом)

Элиминация структуры данных путём сопоставления её со способом конструирования.

```
show :: Pet -> String
show pet = case pet of
  Parrot -> "Parrot"
  Dog name _ -> "Dog: " ++ name
  Cat (CatD name _) -> "Cat: " ++ name
```

- ▶ Шаблоны могут быть вложенными для сложных структур данных

△ **Exhaustiveness** — проверка компилятора, что все варианты конструирования рассматриваются при паттерн-матчинге (старается максимально использовать)

Задачи на питомцев

Декларации

```
data CatT = CatD String Int
data Pet = Cat CatT | Dog String Int | Parrot
```

`todo` Дан питомец, добавьте префикс `"baron"` к его имени

`todo` По питомцу верните возраст

Задачи на питомцев

Декларации

```
data CatT = CatD String Int
data Pet = Cat CatT | Dog String Int | Parrot
```

todo Дан питомец, добавьте префикс "baron" к его имени

todo По питомцу верните возраст



```
baronify :: Pet -> Pet
```

```
baronify pet = case pet of
```

```
  Parrot -> Parrot
```

```
  Dog name age -> Dog ("baron " ++ name) age
```

```
  Cat (CatD name age) -> Cat (CatD ("baron " ++ name) age)
```

Задачи на питомцев

Декларации

```
data CatT = CatD String Int
data Pet = Cat CatT | Dog String Int | Parrot
```

`todo` Дан питомец, добавьте префикс `"baron"` к его имени

`todo` По питомцу верните возраст

```
⚡ baronify :: Pet -> Pet
baronify pet = case pet of
    Parrot -> Parrot
    Dog name age -> Dog ("baron " ++ name) age
    Cat (CatD name age) -> Cat (CatD ("baron " ++ name) age)

⚡ data MaybeInt = Nothing | Just Int
age :: Pet -> MaybeInt
age pet = case pet of
    Parrot -> Nothing
    Dog _ age -> Just age
    Cat (CatD _ age) -> Just age
```

Задачи на рекурсивные типы

```
data Tree = Leaf Int | Node Tree Int Tree
```

- ▶ Рекурсивные типы — способ реализовывать коллекции в Haskell

e.g. `Node (Leaf 1) 2 (Node (Leaf 3) 4 (Leaf 5))`

`todo` Вычислите глубину дерева и поредацируйте на примере

`todo` Вычислите сумму значений и поредацируйте на примере

`todo` Примените к каждому значению в дереве функцию и поредацируйте на примере

Задачи на рекурсивные типы

```
data Tree = Leaf Int | Node Tree Int Tree
```

- ▶ Рекурсивные типы — способ реализовывать коллекции в Haskell

e.g. `Node (Leaf 1) 2 (Node (Leaf 3) 4 (Leaf 5))`

`todo` Вычислите глубину дерева и поредуцируйте на примере

`todo` Вычислите сумму значений и поредуцируйте на примере

`todo` Примените к каждому значению в дереве функцию и поредуцируйте на примере



```
height :: Tree -> Int
```

```
height tree = case tree of
```

```
  Leaf _ -> 1
```

```
  Node l _ r -> 1 + max (height l) (height r)
```

Задачи на рекурсивные типы

```
data Tree = Leaf Int | Node Tree Int Tree
```

- ▶ Рекурсивные типы — способ реализовывать коллекции в Haskell

e.g. `Node (Leaf 1) 2 (Node (Leaf 3) 4 (Leaf 5))`

`todo` Вычислите глубину дерева и поредуцируйте на примере

`todo` Вычислите сумму значений и поредуцируйте на примере

`todo` Примените к каждому значению в дереве функцию и поредуцируйте на примере

```
⚡ height :: Tree -> Int
height tree = case tree of
  Leaf _ -> 1
  Node l _ r -> 1 + max (height l) (height r)
```

```
⚡ transform :: (Int -> Int) -> Tree -> Tree
transform f tree = case tree of
  Leaf x -> Leaf (f x)
  Node l x r -> Node (transform f l) (f x) (transform f r)
```

Содержание

1. data

2. Полиморфные data

3. Списки

4. Всякое разное (⊕)

5. Алгебраические типы данных в других языках (☀)

6. Материалы (☀)

Полиморфные алгебраические типы

- ▶ Тип значений в структуре данных определяется её пользователем
- ▶ Тип структуры данных зависит от типов значений

todo Абстрагируйте `Maybe` по типу содержимого

```
data MaybeInt = Nothing | Just Int
age :: Pet -> MaybeInt
```

todo Какие типы будут у конструкторов данных?

todo Закодируйте полиморфное бинарное дерево и функцию трансформации значений

Полиморфные алгебраические типы

- ▶ Тип значений в структуре данных определяется её пользователем
- ▶ Тип структуры данных зависит от типов значений

todo Абстрагируйте `Maybe` по типу содержимого

```
data MaybeInt = Nothing | Just Int
age :: Pet -> MaybeInt
```

todo Какие типы будут у конструкторов данных?

todo Закодируйте полиморфное бинарное дерево и функцию трансформации значений

```
⚡ data Maybe a = Nothing | Just a
age :: Pet -> Maybe Int
```

Полиморфные алгебраические типы

- ▶ Тип значений в структуре данных определяется её пользователем
- ▶ Тип структуры данных зависит от типов значений

todo Абстрагируйте `Maybe` по типу содержимого

```
data MaybeInt = Nothing | Just Int
age :: Pet -> MaybeInt
```

todo Какие типы будут у конструкторов данных?

todo Закодируйте полиморфное бинарное дерево и функцию трансформации значений

```
⚡ data Maybe a = Nothing | Just a
age :: Pet -> Maybe Int
```

```
⚡ Nothing :: Maybe a
Just :: a -> Maybe a
```

Полиморфные алгебраические типы

- ▶ Тип значений в структуре данных определяется её пользователем
- ▶ Тип структуры данных зависит от типов значений

todo Абстрагируйте `Maybe` по типу содержимого

```
data MaybeInt = Nothing | Just Int
age :: Pet -> MaybeInt
```

todo Какие типы будут у конструкторов данных?

todo Закодируйте полиморфное бинарное дерево и функцию трансформации значений

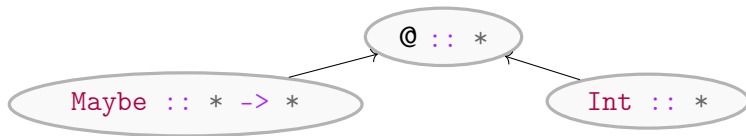
```
⚡ data Maybe a = Nothing | Just a
age :: Pet -> Maybe Int
```

```
⚡ Nothing :: Maybe a
Just :: a -> Maybe a
```

```
⚡ data Tree a = Leaf a | Branch (Tree a) a (Tree a)
transform :: (a -> b) -> Tree a -> Tree b
transform f (Leaf x) = Leaf (f x)
transform f (Branch l x r) = Branch (transform f l) (f x) (transform f r)
```

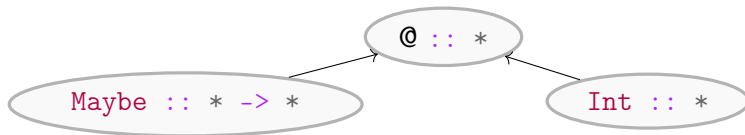
Система кайндов

- ▶ Теперь у нас есть аппликация на уровне типов
- ▶ Нужна система типов над типами для контроля количества типовых аргументов



Система кайндов

- ▶ Теперь у нас есть аппликация на уровне типов
- ▶ Нужна система типов над типами для контроля количества типовых аргументов

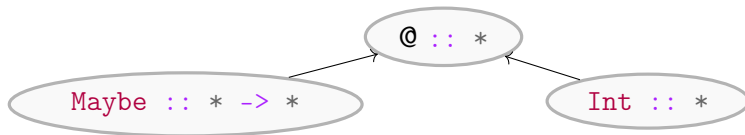


`todo` Определите кайнд конструктора типа

```
data Tree a = Leaf a | Branch (Tree a) a (Tree a)
```

Система кайндов

- ▶ Теперь у нас есть аппликация на уровне типов
- ▶ Нужна система типов над типами для контроля количества типовых аргументов



todo Определите кайнд конструктора типа

```
data Tree a = Leaf a | Branch (Tree a) a (Tree a)
```

todo Определите кайнд конструктора типа

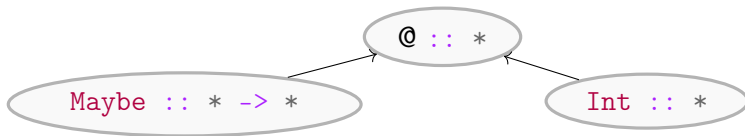
```
data Free f a = Pure a | Free (f (Free f a))
```

todo Определите кайнд конструктора типа

```
data HFree f a = Pure a | HFree (f (HFree f) a)
```

Система кайндов

- ▶ Теперь у нас есть аппликация на уровне типов
- ▶ Нужна система типов над типами для контроля количества типовых аргументов



todo Определите кайнд конструктора типа

```
data Tree a = Leaf a | Branch (Tree a) a (Tree a)
```

todo Определите кайнд конструктора типа

```
data Free f a = Pure a | Free (f (Free f a))
```

todo Определите кайнд конструктора типа

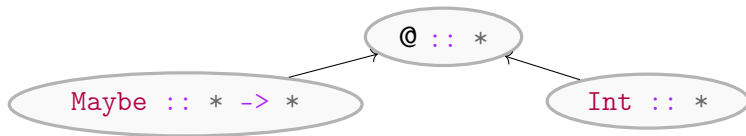
```
data HFree f a = Pure a | HFree (f (HFree f) a)
```



```
Tree :: * -> *
```

Система кайндов

- ▶ Теперь у нас есть аппликация на уровне типов
- ▶ Нужна система типов над типами для контроля количества типовых аргументов



todo Определите кайнд конструктора типа

```
data Tree a = Leaf a | Branch (Tree a) a (Tree a)
```

todo Определите кайнд конструктора типа

```
data Free f a = Pure a | Free (f (Free f a))
```

todo Определите кайнд конструктора типа

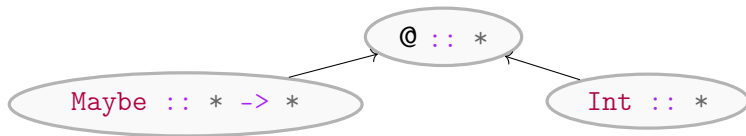
```
data HFree f a = Pure a | HFree (f (HFree f) a)
```

⚡ `Tree :: * -> *`

⚡ `Free :: (* -> *) -> * -> *`

Система кайндов

- ▶ Теперь у нас есть аппликация на уровне типов
- ▶ Нужна система типов над типами для контроля количества типовых аргументов



todo Определите кайнд конструктора типа

```
data Tree a = Leaf a | Branch (Tree a) a (Tree a)
```

todo Определите кайнд конструктора типа

```
data Free f a = Pure a | Free (f (Free f a))
```

todo Определите кайнд конструктора типа

```
data HFree f a = Pure a | HFree (f (HFree f) a)
```

```

⚡ Tree :: * -> *
⚡ Free :: (* -> *) -> * -> *
⚡ HFree :: ((* -> *) -> * -> *) -> * -> *
  
```

Содержание

1. data
2. Полиморфные data
3. Списки
4. Всякое разное (🌐)
5. Алгебраические типы данных в других языках (☀️)
6. Материалы (☀️)

Списки

- ▶ Список — это дерево с одной веткой

```
data List a = Nil | Cons a (List a)
```

- ▶ Стандартный список использует операторы

```
data [] a = [] | (:) a ([] a)
```

- ▶ Есть синтаксический сахар `[] a` $\equiv$ `[a]` и `[1, 2, 3]` $\equiv$ `1 : 2 : 3 : []`

`todo` Реализуйте безопасную функцию взятия головы списка

`todo` Реализуйте функцию, дублирующую элементы с чётными номерами

Списки

- ▶ Список — это дерево с одной веткой

```
data List a = Nil | Cons a (List a)
```

- ▶ Стандартный список использует операторы

```
data [] a = [] | (:) a ([] a)
```

- ▶ Есть синтаксический сахар `[] a` $\equiv$ `[a]` и `[1, 2, 3]` $\equiv$ `1 : 2 : 3 : []`

`todo` Реализуйте безопасную функцию взятия головы списка

`todo` Реализуйте функцию, дублирующую элементы с чётными номерами



```
safeHead :: [a] -> Maybe a
```

```
safeHead xs = case xs of [] -> Nothing; x:_ -> Just x
```

Списки

- ▶ Список — это дерево с одной веткой

```
data List a = Nil | Cons a (List a)
```

- ▶ Стандартный список использует операторы

```
data [] a = [] | (:) a ([] a)
```

- ▶ Есть синтаксический сахар `[] a` $\equiv$ `[a]` и `[1, 2, 3]` $\equiv$ `1 : 2 : 3 : []`

`todo` Реализуйте безопасную функцию взятия головы списка

`todo` Реализуйте функцию, дублирующую элементы с чётными номерами



```
safeHead :: [a] -> Maybe a
```

```
safeHead xs = case xs of [] -> Nothing; x:_ -> Just x
```



```
dupEvens :: [a] -> [a]
```

```
dupEvens = concat . map f . zip [0..]
```

```
where
```

```
  f (i, x) = if odd i then [x] else [x, x]
```

Больше задачек на списки

`todo` Реализуйте разворот списка

`todo` Реализуйте разворот списка эффективно

Больше задачек на списки

todo Реализуйте разворот списка

todo Реализуйте разворот списка эффективно



```
reverse :: [a] -> [a]
```

```
reverse [] = []
```

```
reverse (x:xs) = reverse xs ++ [x]
```

Больше задачек на списки

todo Реализуйте разворот списка

todo Реализуйте разворот списка эффективно

```
⚡ reverse :: [a] -> [a]
reverse [] = []
reverse (x:xs) = reverse xs ++ [x]

⚡ reverse xs = go [] xs
  where
    go :: [a] -> [a] -> [a]
    go acc [] = acc
    go acc (x:xs) = go (x:acc) xs
```


Очевидная тотальность (⚙️)

- △ **Тотальный код** — никогда не расходится (заикливания, исключения)
 - ▶ Но завершаемость проверить нельзя (halting problem)
 - ▶ И исключительные ситуации случаются (ошибки программиста, работа с IO)
- ▶ Вместо этого используем “**очевидную тотальность**”
 - ▶ Моделируем мир типами-суммы, используем exhaustive сопоставления с образцом
 - ▶ Никогда не забываем базу рекурсии
 - ▶ Делаем рекурсивный вызов на структурно меньших термах
- ▶ На ошибке программиста — всё же fail fast¹
- ▶ Выстраиваем чёткую границу с внешним миром²
 - ▶ Данные парсим и преобразуем в структуры данных
 - ▶ Активно используем `newtype` — очень сильно недооценённая в промышленности концепция (но это пока — Java inline classes, Rust tuple structs. . .)
 - ▶ В программе выполнение инвариантов уже гарантируется системой типов

¹<https://en.wikipedia.org/wiki/Fail-fast>

²[Parse, don't validate](#)

Персистентные структуры данных

△ **Эфемерный интерфейс** — изменение структуры данных меняет саму эту структуру

```
xs.append(1)
```

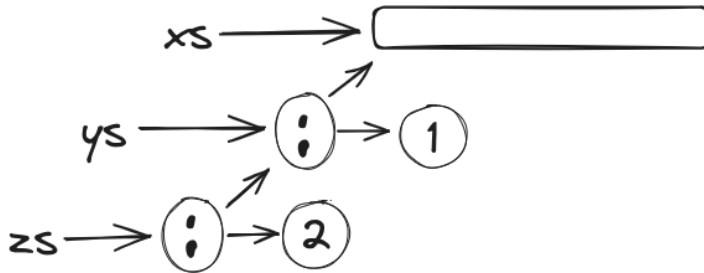
```
xs.append(2)
```

△ **Персистентный интерфейс** — изменение порождает новую структуру, оставляя старую неизменной

```
ys = 1 : xs
```

```
zs = 2 : ys
```

NB В Haskell все структуры данных персистентные



Содержание

1. data
2. Полиморфные data
3. Списки
4. Всякое разное (⊕)
5. Алгебраические типы данных в других языках (☀)
6. Материалы (☀)

Записи (records)

△ Синтаксис для именования полей типов-произведений

► Синтаксис деклараций:

```
data Cat = Cat { catName :: String, catMurlicity :: Int }
```

► Синтаксис создания (и, соответственно, паттерн-матчинга)

```
cat = Cat { catName = "barsik", catMurlicity = 42 }
```

► Генерируются функции для доступа (применение — `catName cat`):

```
catName :: Cat -> String
catMurlicity :: Cat -> Int
```

► Есть синтаксис для функционального обновления структуры:

```
scratch cat = cat { catMurlicity = 1 + catMurlicity cat }
```

⊗ Использование с типами-суммы — плохая практика (не тотальный код)

⊗ Засоряется глобальное пространство имён, поэтому используют префиксы имени типа

⊗ Для удобства работы существует множество расширений в компиляторе³

³https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/records.html

Compile-time типовые обёртки: newtype

- ▶ Если у `data` один конструктор с одним параметром

⇒ То выделять память под объект не обязательно — храним сразу параметр

△ `newtype` — более оптимальный вариант `data` для этого случая

e.g. На этапе компиляции — разные типы, во время исполнения — числа

```
newtype ModuleId = ModuleId Int64
```

```
newtype CourseId = CourseId Int64
```

- ⊕ Разные сущности предметной области должны иметь разные типы (по-хорошему)

```
publish :: ModuleId -> CourseId -> Publisher Result
```

```
main = ... publish courseId moduleId ... -- Compilation error
```

- ⊕ Обеспечение инвариантов значений программы:

```
parseCourseId :: Int64 -> CourseId
```

```
parseCourseId id | isValidCourseId id = CourseId id
                  | otherwise = report error somehow
```

- ⊕ Возможен бесплатный переход между типами обёрток одного и того же

```
intsToIds :: [Int64] -> [CourseId]
```

```
intsToIds = coerce
```

Синонимы типов: type

- ▶ Синонимы ничем не отличаются от соответствующих типов (взаимозаменяемы в коде)
- ▶ Используются для придания типам доменно-специфичных имён:

```
type Name = String
showName :: Name -> String
showName name = "Name: " ++ name
```

- ▶ А также для сокращения записи длинных типов
- ▶ Могут быть полиморфными (иметь типовой параметр)

```
type State a = Map Name a
State Int ≡ Map Name Int
```

Полиморфные типы demystified или система $\lambda\omega$ $(*)$ (☀)

- Подстановка типовых параметров в типы подозрительно напоминает подстановку термов в термы

```
type Pair a b = forall c. (a -> b -> c) -> c
Pair Int Char ≡ forall c. (Int -> Char -> c) -> c
```

- * Это фактически она и есть (только для типов)!

Pair : $* \rightarrow * \rightarrow *$

Pair = $\lambda\tau^* \sigma^*. \forall\gamma. (\tau \rightarrow \sigma \rightarrow \gamma) \rightarrow \gamma$

pair : $\forall\alpha \beta. \alpha \rightarrow \beta \rightarrow \mathbf{Pair} \alpha \beta$

pair = $\Lambda\alpha^* \beta^*. \lambda x^\alpha y^\beta. (\Lambda\gamma^*. \lambda f^{\alpha \rightarrow \beta \rightarrow \gamma}. f \ x \ y)$

fst : $\forall\alpha \beta. \mathbf{Pair} \alpha \beta \rightarrow \alpha$

fst = $\Lambda\alpha^* \beta^*. \lambda p^{\mathbf{Pair} \alpha \beta}. p \ \alpha \ (\mathbf{K} \ \alpha \ \beta)$

Содержание

1. data
2. Полиморфные data
3. Списки
4. Всякое разное (⊕)
5. Алгебраические типы данных в других языках (☀)
6. Материалы (☀)

Типы-произведения в Си (☺)

Декларация типа-произведения

```
struct CatTD {  
    string name;  
    int murlicity;  
};
```

- ▶ Имена типа и конструктора данных автоматически совпадают

Конструирование

```
CatTD createBarsik() {  
    return CatTD{"barsik", 42};  
}
```

Элиминация

```
string getName(CatTD cat) {  
    return cat.name;  
}
```

Типы-суммы в Си (☺)

```

struct Pet {
    enum { CAT; DOG; PARROT } tag;
    union {
        struct { CatT catData } cat;
        struct { Name name } dog;
    } value;
};

if (pet.tag == CAT) {
    printf("Cat: %s", pet.value.cat.catData.name);
} else if (pet.tag == DOG) {
    printf("Dog: %s", pet.value.dog.name);
} else if (pet.tag == PERROT) {
    printf("Perrot");
} else {
    assert(false && "unreachable code"); // No exhaustiveness check
}

```

Типы-суммы в C++⁴ (☺)

```
std::variant<Cat, Dog, Parrot> pet{Cat{"barsik", 42}};

if (std::has_alternative<Cat>(pet)) {
    std::cout << "Cat " << std::get<Cat>(pet).name << std::endl;
} else if (std::has_alternative<Dog>(pet)) {
    std::cout << "Dog " << std::get<Dog>(pet).name << std::endl;
} else if (std::has_alternative<Parrot>(pet)) {
    std::cout << "Parrot" << std::endl;
} else {
    assert(false && "unreachable code"); // No exhaustiveness check
}
```

⁴<https://en.cppreference.com/w/cpp/utility/variant>.

Типы-суммы в ООП языках: пример на Python (☺)

```
class Pet: pass
class Cat(Pet):
    cat: CatT
class Dog(Pet):
    name: Name
class Parrot(Pet): pass

if isinstance(pet, Cat):
    print("Cat " + pet.cat.name)
elif isinstance(pet, Dog):
    print("Dog " + pet.name)
elif isinstance(pet, Parrot):
    print("Parrot")
else:
    assert False, "unreachable code" // No exhaustiveness check
```

Типы-суммы в ООП языках: пример на Kotlin (☺)

- ▶ Тюленезированные классы (wat?) заставляют определять всех наследников в пределах модуля
- ▶ Так у компилятора достаточно информации для проверки exhaustiveness

```
sealed class Pet
class Cat(val cat: CatT) : Pet
class Dog(val name: Name) : Pet
class Parrot : Pet

when (pet) {
    is Cat -> print("Cat: " + pet.cat.name)
    is Dog -> print("Dog: " + pet.name)
    is Parrot -> print("Parrot")
}
```

Содержание

1. data
2. Полиморфные data
3. Списки
4. Всякое разное (⊕)
5. Алгебраические типы данных в других языках (☀)
6. Материалы (☀)

Что посмотреть в транспорте

- ▶ Parse, don't validate
- ▶ (youtube) Тагир Валеев — Pattern matching в Java и его воображаемые друзья
- ▶ The algebra (and calculus!) of algebraic data types